Paper Critique: LoRA: Low-Rank Adaptation of Large Language Models

Full Name: Mathis DOUTRE
Student ID: 904-04-8015
Paper #: 5
Title : LoRA: Low-Rank Adaptation of Large Language Models

---

1 Problems Adressed

The paper LoRA, by Hu et al. (2021)[1] tackles the practical challenges of fine-tuning very large pre-trained language models for specific problems. As models became larger (such as GPT-3 with 175 billion parameters), **full fine-tuning** (updating all model weights for each new task) became impractical due to computational and storage costs. Fine-tuning a 175B-parameter model for every task means deploying a separate model instance of that size for each use case, which is increasingly expensive in terms of memory and storage. As an example, a single GPT-3 checkpoint in FP16 is 350GB, maintaining 100 fully fine-tuned variants would require 35 TB. This creates a bottleneck: Storing hundreds of such large models is simply infeasible.

Another problem the paper tackles is the fine-tuning **efficiency barrier**. Updating billions of weights not only consumes enormous GPU memory (gradients and optimizer states), but also demands time-consuming training. Fine-tuning GPT-3 "requires 780 GB of GPU memory" (65B model, 16-bit), illustrating the hardware barrier to adapting such models. Even if one has the resources to fine-tune a single large model, doing so for many tasks multiplies the cost.

Researchers have proposed parameter-efficient adaptation methods to address these issues, but those methods came with trade-offs. One class of methods is inserting small adapter layers into the network (Houlsby et al., 2019 [2]), so that only the adapter parameters (actually representing a small fraction of the model) are trained for each task. However, these techniques "often introduce inference latency"([1]) by adding extra layers in the forward pass. Another approach is prefix-tuning or prompt-tuning, which keeps the model weights fixed and instead learns task-specific embeddings or "prompts" (Li and Liang, 2021 [3]). This usually reduce trainable parameters but may "reduce the model's usable sequence length"([1]) (since prompt tokens consume input space) and can be less effective than fine-tuning. Indeed, prior efficient tuning methods often failed to reach the full fine-tuning accuracy, indicating in practice a quality vs. efficiency trade-off. Bias-only tuning (Zaken et al., 2021 [4]) or

fine-tuning only the last layers reduces the overhead cost but often underperforms full tuning on complex tasks.

The core problem the paper addresses is: **How can we adapt large language models to new tasks with fewer trainable parameters and lower compute costs, while maintaining performance?** The authors specifically target the scenario of massive Transformer-based LLMs where duplicating the entire model for each task is not practical. They seek a solution that lowers memory and computation overhead for fine-tuning, without sacrificing model quality or increasing inference time. The paper addresses the bottleneck of deploying LLMs at scale.

2 New Idea and Strengths

LoRA (Low-Rank Adaptation) introduces an elegant solution to those problems : instead of adjusting all of a model's parameters when training for a specific task, it freezes the original weights and learns a **small additive update** for each weight matrix, constrained to be **low-rank**. This idea is motivated by earlier findings that the changes needed to fine-tune a large model might lie in a **low-dimensional subspace** (Aghajanyan et al., 2020 [5]). In other words, the authors propose that "the change in weights during model adaptation has a low intrinsic rank"[1], even if the weight matrices themselves are high-rank and high-dimensional.

LoRA implements this by representing the weight update ΔW as a product of two much smaller matrices: B and A (of dimension $d \times r$ and $r \times d$ respectively, with $W$of size $d \times d$), as shown in Figure 1. Here r is a chosen rank much smaller than d. During fine-tuning, the original weight W_0 remains unchanged, and only the small matrices A and B are trained, yielding an updated weight $W_{\text{ft}} = W_0 + \Delta W = W_0 + B \times A$. This low-rank parameterization reduces drastically the number of trainable parameters: instead of $d \times d$ parameters to learn for ΔW, LoRA needs only $2dr$ parameters (which for small r is orders of magnitude smaller).

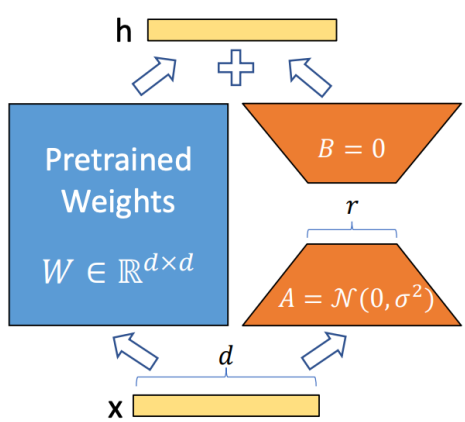

Figure 1: LoRA approach: W is the frozen pre-trained matrix; B and A are trainable low-rank factors with $\Delta W = BA$. Only A, B are updated during fine-tuning and merged into W for inference.

Strengths and innovations of LoRA:

By tuning low-rank matrices instead of the full matrix, LoRA **reduces the number of trainable parameters** by many orders of magnitude. For GPT-3, LoRA uses $17M$ trainable parameters, which is **10,000x fewer than the base model**. The resulting deltas are only tens of megabytes (e.g. 35 MB) as opposed to hundreds of gigabytes for a full model. This allow hundreds of task-specific models to be stored with minimal overhead. Freezing most weights also **reduces training memory** (e.g. in their GPT-3 setup, from $1.2TB$ to $350GB$).

LoRA **reduces trainable parameters and compute**: with most weights frozen, back-propagation and optimizer state are needed only for the small A and B matrices. Because 99.99% of parameters remain fixed, gradients and optimizer buffers are not computed and not stored for them. In GPT-3 experiments, this resulted to up to a 3× reduction in GPU memory usage compared with full fine-tuning, making large-model adaptation possible on more modest hardware and enabling more parallel runs. LoRA *"lowers the hardware barrier to entry"* for fine-tuning large models [1].

After training, **LoRA merges the low-rank update into the base weights**, $W_{ft} = W_0 + BA$, so inference uses a standard weight matrix with no added latency and behavior **equivalent** to full fine-tuning. In contrast, methods like prefix-tuning [3] or adapters add tokens or layers at inference, increasing compute and potentially slowing throughput.

LoRA often **matches or surpasses full fine-tuning** despite using far fewer trainable parameters. On GLUE with RoBERTa/DeBERTa and on GPT-2/GPT-3 across tasks such as WikiSQL, MultiNLI, and SAMSum, LoRA achieves comparable or better accuracy. As shown in Table 2, a GPT-3 LoRA model with only 4.7M trainable parameters reaches higher accuracy than full fine-tuning on WikiSQL (73.4 vs. 73.8), matches or exceeds performance on MultiNLI (91.7 vs. 89.5), and achieves the best ROUGE scores on SAMSum. LoRA also outperforms other efficient tuning methods (BitFit, prefix-based, adapters) while remaining orders of magnitude smaller. These results support the claim that **effective fine-tuning updates are low rank**.

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($Adapter^H$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($Adapter^H$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

Figure 2: LoRA compared with full fine-tuning and other parameter-efficient methods on GPT-3 across WikiSQL, MultiNLI, and SAMSum (from Hu et al., 2021 [1]).

LoRA **scales well with model and task complexity**. In Figure 3, accuracy on WikiSQL and MNLI increases with rank (i.e., more trainable parameters) and exceeds other methods for the same parameter budget. LoRA achieve strong results at low ranks and continuing to improve as rank grows. Others methods often plateau or degrade when given more trainable parameters. This indicates LoRA **uses limited parameters efficiently**.

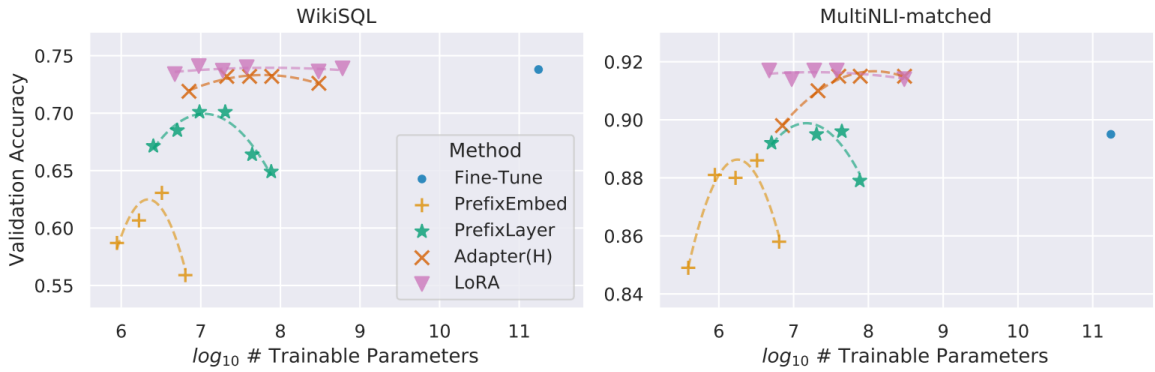

Figure 3: Validation accuracy vs. number of trainable parameters for GPT-3 (175B) on WikiSQL and MNLI-matched. LoRA attains higher accuracy at comparable budgets and scales smoothly with rank (from Hu et al., 2021 [1]).

With the base model frozen, task-specific LoRA modules (B, A pairs) can be swapped during inference with negligible overhead, enabling a library of fine-tuned models on a single checkpoint.
This **modularity** also means LoRA composes with other strategies: for example, using prefix-tuning prompts alongside LoRA-modified weights, or with Compacter's Kronecker-factored adapters [6] to further cut trainable parameters [1].

LoRA is **simple** to implement and requires no architectural changes, making it easy to reason about. An open-source PyTorch implementation further lowers the adoption barrier [7].

3 Weaknesses and Suggestions

Choosing the rank r is critical: very low ranks can underfit task-specific changes, while high ranks reduce LoRA's parameter savings. Although the paper states low ranks (1-8) are often enough and returns diminish for high values, some methods show non-monotonic behavior at larger budgets [8]. A possible future work is automatic rank selection to a parameter or memory budget, reducing manual tuning and adapting to the task complexity.

In the reported implementation, LoRA is **applied primarily to attention weights**, with MLP layers, LayerNorms, and biases **kept frozen** for simplicity and parameter efficiency. This restriction **may limit performance** on tasks that rely on adaptations in the frozen components. A natural extension is to evaluate low-rank adaptations beyond attention. For example selectively applying LoRA to MLP blocks, LayerNorm parameters, or biases, and to measure when such extensions yield meaningful gains compared to the added parameter cost.

LoRA is **optimized for single-task deployment**: when updates are merged ($W_{ft} =$

$W_0 + BA$), **only one adapter (A,B) can be active**, preventing a single forward pass from batching examples that require different adapters. The alternative is to **avoid merging** and select the adapter on the fly (meaning computing $W_0x + BAx$) which introduces some **inference overhead**.

Because the base weights remain frozen, **noise from pretraining can persist** and may be amplified in the adapted model (this is called **"catastrophic inheritance"**). Mitigations strategies include adding regularization to the LoRA updates to constrain deviation from the base model: BA-LoRA (Chang et al., 2024 [9]), aims to preserve knowledge while reducing noise.

LoRA's effectiveness **depends on hyperparameters and initialization**. Hu et al. initializes $B = 0$ and A randomly so $\Delta W = BA = 0$ at step 0, starting the tuning from the original weights. However, this can slow convergence in short fine-tuning runs (Huang and Balestriero (2024) [10]). Implementations also use a scaling factor α to control the magnitude of BA; poor choices can harm the optimization. Recent variants (ALLoRA [10]) replace manual scaling and dropout with adaptive, learning rates that normalize updates, improving efficiency.

Furthermore, **extending LoRA beyond NLP** seems promising because many domains face similar fine-tuning costs. Research reports applications to CNNs, speech models, and scientific pretrained models, suggesting the approach generalizes across modalities [11]. Overall, LoRA addresses a central fine-tuning cost with a simple low-rank update mechanism.

References

[1] EdwardJ. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Lu Wang, Sheanand Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. URL https://arxiv.org/abs/2106.09685.

[2] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. *arXiv preprint arXiv:1902.00751*, 2019. URL https://arxiv.org/abs/1902.00751.

[3] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021. URL https://arxiv.org/abs/2101.00190.

[4] Elad BenZaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *arXiv preprint arXiv:2106.10199*, 2021. URL https://arXiv.org/abs/2106.10199.

[5] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *arXiv preprint arXiv:2012.13255*, 2020. URL https://arxiv.org/abs/2012.13255.

[6] RabeehKarimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers. In *Advances in Neural Information Processing Systems (NeurIPS) 2021*, 2021. URL https://arXiv.org/abs/2106.04647.

[7] Microsoft. Lora: Code for low-rank adaptation of large language models (github repository). https://github.com/microsoft/LoRA, 2025.

[8] Rohan Sukumaran, Aarash Feizi, Adriana Romero-Soriano, and Golnoosh Farnadi. Fairlora: Unpacking bias mitigation in vision models with fairness-regularized low-rank adaptation. In *ICLR 2025 (OpenReview)*, 2025. URL https://openreview.net/forum?id=pB3KeBCnQs. Under review, OpenReview preprint.

[9] Yupeng Chang, Yi Chang, and Yuan Wu. Ba-lora: Bias-alleviating low-rank adaptation to mitigate catastrophic inheritance in large language models. *arXiv preprint arXiv:2408.04556*, 2024. URL https://arxiv.org/abs/2408.04556.

[10] Hai Huang and Randall Balestriero. Allora: Adaptive learning rate mitigates lora fatal flaws. *arXiv preprint arXiv:2410.09692*, 2024. URL https://arXiv.org/abs/2410.09692.

[11] Menglin Yang, Jialin Chen, Yifei Zhang, Jiahong Liu, Jiasheng Zhang, Qiyao Ma, Harshit Verma, Qianru Zhang, Min Zhou, Irwin King, and Rex Ying. Low-rank adaptation for foundation models: A comprehensive review. *arXiv preprint arXiv:2501.00365*, 2025. URL https://arXiv.org/pdf/2501.00365.

Paper Critique: Resource-Efficient Transformer Pruning for Finetuning of Large Models

Full Name: Mathis DOUTRE
Student ID: 904-04-8015
Paper #: 6
Title : Resource-Efficient Transformer Pruning for Finetuning of Large Models

---

1 Problems Adressed

Ilhan et al. (2024) address the high GPU-memory cost of fine-tuning large transformers. State-of-the-art models (e.g., ViTs, BERT, GPT) contain billions of parameters, enabling strong generalization but making fine-tuning memory-intensive. For instance, a 70B-parameter LLaMA-2 needs 14 GB for inference and 70 GB for 16-bit fine-tuning. These ressources are out of reach for many users.

To reduce memory load, many efficient fine-tuning techniques have been explored, but each has limits. **Quantization** reduces model size but is not task-aware and may lose information, especially with outlier weights [2, 3]. Quantization is typically applied post-training during inference, so they do not reduce fine-tuning memory.

Adapter-based methods train small modules while freezing the backbone. For example, **LoRA** adds low-rank adapters [4], and prefix-tuning learns task-specific prefix tokens [5]. However, these methods still require loading the full backbone on GPU during training, leaving the memory bottleneck unsolved for very large models.

Model pruning is another approach: **Pruning** removes weights not needed for the target task and has been effective for shrinking models and speeding inference. However, it creates a trade-off. **Pruning before the adaptation** (*Pre-fine-tuning pruning*) risks discarding weights needed to learn the task, often reducing downstream accuracy [6, 7]. **Pruning after the fine-tuning** (*Post-fine-tuning*) better preserves accuracy but still requires loading and updating the entire model during the initial fine-tuning. Indeed, prior research found pruning gives the best results when done after fine-tuning.

In summary, the paper addresses two main problems:

Training memory challenge: Fine-tuning transformers often exceeds the memory of GPUs. The goal is to reduce memory during training without sacrificing accuracy.

Limitations of existing methods: Prior approaches either compromise task performance or still require holding the full model in memory. The paper targets this gap by enabling resource-aware pruning during fine-tuning to maintain accuracy while lowering memory cost.

2 New Idea and Strengths

The authors introduce RECAP (REsource and ComputAtion-efficient Pruning) [1], a method that integrates pruning into the fine-tuning process to lower GPU memory requirements while maintaining accuracy. The key idea is to bridge the gap between pre- and post-finetune pruning by performing an iterative fine-tuning that only ever operates on a subset of the model at any given time. This approach is illustrated in the Figure 1, which contrasts RECAP against conventional strategies: pure **pre-finetune pruning** (which can lead to "suboptimal pruning" by itself) and pure **post-finetune pruning** (which faces a "resource bottleneck" of needing the full model in GPU memory). **RECAP's design** uses both CPU and GPU resources in tandem to avoid those pitfalls.

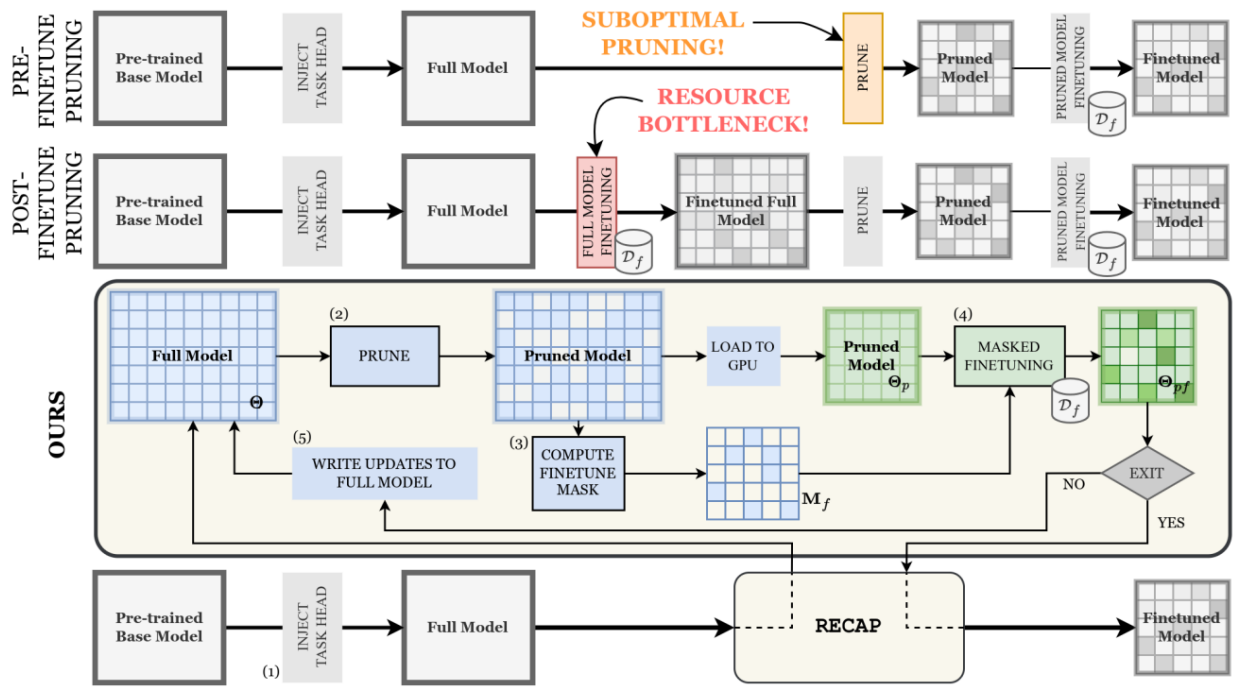

Figure 1: System architecture of RECAP. Components in blue are done on CPU, and those in green are done on GPU (from Ilhan et al., 2024 [1]).

How RECAP works:

RECAP runs an iterative three-stage pipeline (pruning, fine-tuning, updating), repeated until the model is sufficiently tuned.

Stage 1, Pruning (CPU): The method processes one model chunk at a time: the full model stays in CPU memory, and only the active chunk is evaluated for pruning for each iteration. Weights are organized into groups (e.g., entire attention heads or MLP neurons). Group importance is scored with a first-order Taylor criterion using gradients, estimating the loss increase if the group were removed. Groups are ranked by this importance metric, the algorithm **prunes** away the **least important** weight groups. For example, RECAP can drop entire attention heads or hidden units with low-impact. This stage runs on CPU only and outputs a pruned chunk for the next step.

Stage 2, Fine-tuning (on GPU, with masked updates): During this training phase, only the pruned smaller model is on the GPU. RECAP then performs *Fisher-informed masked updates*: it trains a selected **subset of parameters** (higher estimated impact from gradients) while freezing the rest.
This has **two benefits**: *(i) Memory savings:* by updating only (for example) 25% of the remaining weights, the GPU only needs to allocate gradient and optimizer tensors for that fraction, significantly cutting memory overhead. *(ii) Training stability:* the model prevents the pruned regions from quickly regaining importance by not adjusting every weight, making pruning more stable.

Stage 3, Updating (CPU coordination): After fine-tuning the pruned smaller model, the GPU weights are transfered into the CPU model. RECAP then repeats the *prune* $\rightarrow$ *load* $\rightarrow$ *fine-tune* $\rightarrow$ *update* cycle for a certain number of rounds or until a stopping criterion is met.

This process covers the model without ever loading it fully on the GPU. The result is a model pruned to the target sparsity and adapted to the task, achieved within GPU memory limits.

Strengths and innovations of RECAP:

At any given time, only a **fraction of the weights** are on the GPU, and only **a fraction** of those weights are **updated. Memory overhead drops** across weights, activations, gradients, and optimizer states. The authors report up to **65%** lower GPU memory than existing pruning or adapter methods. In ViT fine-tuning, RECAP uses roughly **one-third** of the memory of full fine-tuning or post-pruning fine-tuning. Figure 2 shows RECAP on the top-left of the accuracy–memory curve: it can exceed other methods at the same memory budget or match their accuracy with far less memory. In contrast, approaches like Post-FT pruning or Movement pruning still load the entire model on GPU, so their memory matches full fine-tuning.

Despite aggressive pruning and partial updates, RECAP **maintains accuracy close to full-model fine-tuning**. On CIFAR-100 classification and Cityscapes/KITTI segmentation, RECAP reaches similar mIoU while using 30–40% less GPU memory. On GLUE with BERT-base, it matches full fine-tuning and exceeds naive pre-fine-tuning pruning at the same sparsity.

RECAP is task-agnostic and model-agnostic: it groups transformer weights into structured

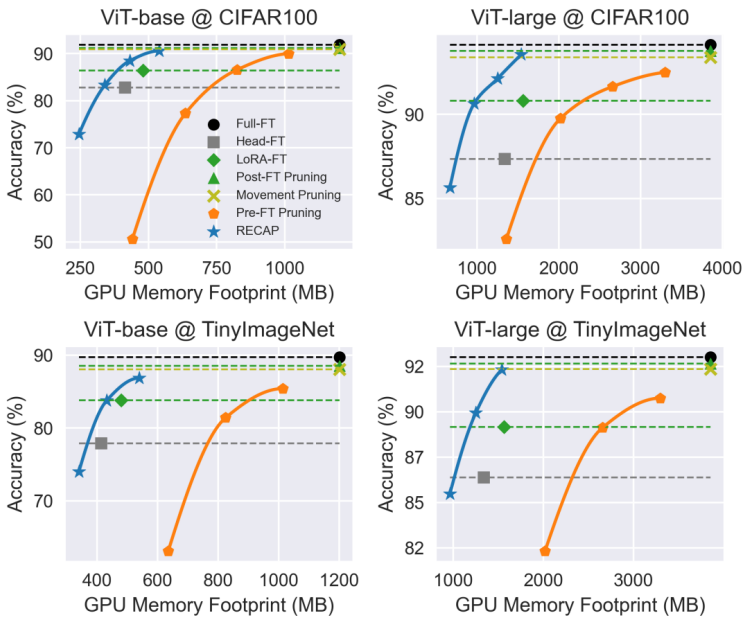

Figure 2: Accuracy vs. GPU memory for ViT-Base/Large: RECAP matches or exceeds baselines with equal memory, or achieves similar accuracy with less memory; post–fine-tuning and movement pruning peak at Full-FT memory (from Ilhan et al. [1]).

units and requires no task-specific architecture changes. Evaluations on computer vision (ViTs for classification and segmentation) and NLP (BERT on GLUE), indicate broad applicability. The same procedure should extends to other transformer models, including larger language models, making it a good option for resource-constrained fine-tuning.

Unlike pipelines that assume a full-model fine-tune before pruning, **RECAP does not require full-model fine-tune**. We can start from a pretrained model and apply RECAP on limited hardware. This enables users with modest GPUs to fine-tune large models without loading the entire model at once.

3 Weaknesses and Suggestions

RECAP introduces **additional steps** (iterative rune, mask, fine and tune) compared to standard fine-tuning; this **increase total training time**.

As the Figure 3 shows RECAP reduces GPU memory but takes longer than full FT, reflecting a **trade-off** of time for memory. Future work could optimize the pipeline by overlapping CPU computations with GPU training.

RECAP assumes the **full model resides in CPU memory** when it is not on the GPU. While RAM is usually larger and cheaper than GPU VRAM, very large models can still exceed available host memory (e.g., we need 140 GB just to store a 70B-parameter model in FP16). The paper mainly evaluates ViT-Large and BERT-base–scale models (hundreds of millions of parameters), so practical scalability to larger tens-of-billions-parameter models remains unproven.

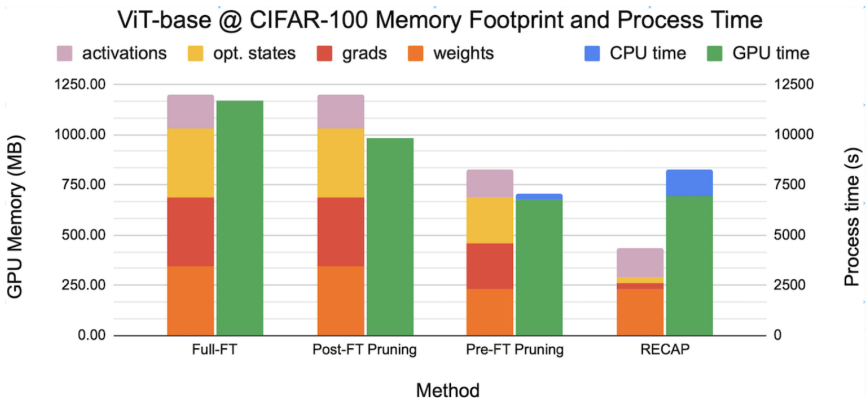

Figure 3: Breakdown of GPU memory footprint (left bars) and process time (right bars) for ViT-base at CIFAR100 (from Ilhan et al. [1]).

RECAP saves memory mainly by reducing on-GPU weights and limiting gradient storage, but in tasks with large inputs or high-resolution features, **activations can dominate**. In the paper's segmentation study (Mask2Former on Cityscapes), activations accounted for **57% of total memory**. An adapter-based approach used **less memory** by freezing the backbone and skipping its gradients, at the cost of 0.5–1 mIoU lower accuracy. This suggests RECAP's advantage **narrows** when activation size is the primary constraint, not model size. This limits its applicability to high-resolution tasks like semantic segmentation or video. Future work could include freezing the most activation-heavy layers, to control memory use when activations dominate.

Because **pruning reduces capacity**, RECAP can fall a little behind full fine-tuning. On Cityscapes, a RECAP-pruned *Swin-Large* reaches **79.5 mIoU versus 82.7 for full fine-tuning**. This is because the partial updates may miss interactions that benefit from joint optimization of all weights. **Suggestions:** We could run a brief fine-tune at the end if we have access to the necessary ressources. This assumes the RECAP-pruned model is close enough to a good optimum that full fine-tuning can resume from it under the same conditions.

The method is **designed for transformer-based models**, with pruning structured around heads and feed-forward units. This covers many modern architectures, but its applicability to other types like CNNs is unexplored. While transformers dominate many domains, this limitation is worth being noted. **Suggestions:** Future work could test RECAP's ideas on other architectures, possibly requiring new grouping strategies. Applying them to larger transformer-based models (e.g., GPT-scale) would help clarify scalability and the issues from longer sequence lengths and higher activation memory.

References

[1] Fatih Ilhan, Gong Su, Selim Furkan Tekin, Tiansheng Huang, Sihao Hu, and Ling Liu. Resource-efficient transformer pruning for finetuning of large models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/papers/Ilhan_Resource-Efficient_Transformer_Pruning_for_Finetuning_of_Large_Models_CVPR_2024_paper.pdf.

[2] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS) 2022*, 2022. URL https://arXiv.org/abs/2208.07339.

[3] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. URL https://arXiv.org/abs/2210.17323.

[4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. URL https://arxiv.org/abs/2106.09685.

[5] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021. URL https://arxiv.org/abs/2101.00190.

[6] Trevor Gale, Erich Elsen, and Sara Hooker. The state of sparsity in deep neural networks. *arXiv preprint arXiv:1902.09574*, 2019. URL https://arXiv.org/abs/1902.09574.

[7] Michael Zhu and Suyog Gupta. To prune, or not to prune: Exploring the efficacy of pruning for model compression. 2017. URL https://arXiv.org/abs/1710.01878.